Movie Revenue Prediction

Eric Wu, Matteo Martone, Jeremy Long, Zheng Dong

Introduction

Github repository - https://github.com/Eric7895/Movie_Revenue_Web_Application

Background

The film industry faces big financial risks, with studios investing millions in production and marketing without a guarantee of success. High stakes and unpredictable market conditions mean that having a reliable way of predicting movie revenues is crucial. When factors like changing viewer tastes and economic challenges come into play, older methods often fail to capture the ever-changing nature of the industry, which can lead to expensive mistakes.

The move toward more data-driven methods shows the need to cut these risks while making planning better. Using historical data and modern tools helps turn complex market signals into practical insights. This change not only aims to improve revenue predictions but also to give a clearer picture of the factors behind financial success, ultimately supporting better planning and investment decisions.

Goal and Objective

Our goal is to develop a robust, data-driven model that predicts revenue over a half-year window for future movie releases with near state-of-the-art accuracy. We will achieve this by evaluating the influence of pre-release metadata on movie revenues, adjusting for dollar inflation using the Consumer Price Index (CPI) to standardize revenue figures across different time periods, and ensuring a proper time-ordered approach where only past data is used to predict future events, thereby preventing data leakage. Additionally, we aim to identify key indicators from the metadata that have a significant impact on the financial success of a movie.

In addition to building our predictive model, we plan to deploy it in a user-friendly manner. To do so, we will set up a SQL database connection to support our FASTAPI application, design the frontend using CSS, HTML, and JavaScript, and deploy the predictive model as an interactive web feature.

Prior Work

Research on movie revenue prediction has gained traction over the past decade, drawing from a wide range of machine learning and econometric methods. Much of the early work focused on linear regression and decision trees using metadata such as budget, genre, release date, and star power. For instance, Zhang et al. (2009) had a simplified approach using mainly decision trees to segment films into high- and low-performing categories. Their approach emphasized interpretability and offered producers a rule-based system for understanding which combinations of factors (e.g., genre, seasonality, sequel status) increased the likelihood of financial success. This paper also introduced methodology for computing monetary features into real dollar amounts using a Consumer Price Index translation. While effective for categorical classification, the model did not

estimate continuous revenue values, and therefore produced unsatisfactory results(59.73% R^2)—highlighting a gap in regression-based modeling that our study seeks to address.

Sharda and Delen (2006) provided foundational research for using data mining techniques to forecast box office success. Their comparative study evaluated decision trees, neural networks, and linear regression models, concluding that neural networks significantly outperformed traditional statistical methods, particularly in capturing the non-linear interactions among features such as budget, MPAA rating, release timing, and star score.

We wanted to build on this foundation to build a project that combines the most successful elements of these other projects to create an optimal solution that would lead to better results. Importantly, it also includes actor and director scoring mechanisms based on historical box office performance, introducing domain-specific enhancements to the feature set. These design choices informed our own approach, particularly in how we engineered similar role-based features to reflect past project success and career consistency, such as star score.

Our approach emphasized robust feature engineering (e.g., primary writer/director extraction, trailer engagement metrics), regularization techniques to address multicollinearity, and cross-validation to benchmark competing models. By extending these methodologies with improved runtime-budget interactions and leveraging more granular genre encoding, we aimed to close gaps in past approaches and improve generalizability to upcoming releases.

Method

Data Collection and Feature Engineering

Starting with data collection, our group chose to use the official IMDB non-commercial datasets. This source is updated daily and is highly compatible with scraping tools. From these official datasets, we obtained many primary attributes such as title, directors, writers, actors, IMDB rating, number of votes, release date, runtime, and genres.

In addition to the IMDB dataset, we also found a secondary dataset on GitHub that includes supplementary variables such as trailer views, trailer likes, revenue, budget, keywords, and more. By incorporating this dataset, we avoided the ongoing hassle of sourcing box office data manually. However, since this dataset is static, we lose the ability to refresh it using scraper tools. To implement a complete Extract, Transform, Load (ETL) pipeline, we would need to identify a legal and up-to-date source for scraping box office numbers and use the YouTube API—combined with natural language processing (NLP) techniques—to extract trailer-related metrics such as views and likes.

Additionally, we attempted to scrape actor, director, and writer "billboards" for feature engineering purposes. We ensured that the sources used were recently updated and broadly representative. For directors and writers, we used an IMDb list created by a user, which was most recently updated on April 15, 2025, and has received 98,000 views. For actors, we used The Numbers billboard to collect actor information and corresponding star scores. We acknowledge that these billboards—especially those created by users—may contain biases, and addressing this bias is a potential area for future improvement.

Moving on to data processing, we removed all movies with null values and still retained a relatively sufficient number of records for model training. We also eliminated duplicate entries by using a composite primary key consisting of the movie title and release date. After cleaning the data, we began feature engineering. Our encoding approaches for each categorical variable are as follows:

- **Director and Writer**: We counted the total number of famous directors and writers associated with each movie, based on their appearance in our billboard lists.
- **Actor**: In addition to counting the number of famous actors, we also computed the cumulative star score for each movie.

- **Genre**: Since the genre field contains comma-separated strings, we extracted only the primary genre (i.e., the first listed) and created dummy variables based on these values.
- **Release Date**: We created two binary variables—is_holiday_release and is_competitive_month. The holiday variable flags releases near major holidays (e.g., Christmas, Thanksgiving, Halloween, Independence Day, Memorial Day, Labor Day, Valentine's Day, and Chinese New Year), while the competitive month variable flags historically high-grossing periods (May, June, July, November, and December).
- **Production Companies**: We created a binary variable, production_popular, to indicate whether a movie was produced by a major company. Our list includes: Warner Bros, Universal Pictures, Walt Disney, Columbia Pictures, 20th Century Fox, Paramount, Marvel Studios, New Line, DreamWorks, Lionsgate, MGM, Sony Pictures, Lucasfilm, Pixar, Legendary, A24, Focus Features, Amblin, Working Title, StudioCanal, and Castle Rock. In the future, we may use string similarity measures (e.g., cosine similarity) to better capture variations in company names.
- **Languages**: We selected the top five most common languages and created dummy variables for each. All other languages were grouped into an "Other Languages" category. In the future, we may consider combining these into a single variable to reduce multicollinearity.
- **Keywords**: We used the pretrained BERT model 'all-mpnet-base-v2' to embed each keyword into high-dimensional vectors. We then applied Principal Component Analysis (PCA) to reduce the dimensionality, selecting the first principal component. We ensured that the first eigenvalue explained at least 40% of the variance. Using a model like BERT helps preserve the semantic meaning of keywords.

By incorporating these features, we greatly expanded our pool of predictors and enhanced the expressiveness of our dataset for model training.

Star Score

The Star Score feature aims to quantify the impact of cast, directors, and writers on a movie's potential box-office performance by leveraging external "fame" data. In this section, we outline a general approach to computing and integrating Star Score into our modeling pipeline. As more detailed information becomes available—such as finalized name lists, precise scoring metrics from thenumbers.com, and actor metadata—this section will be updated and refined.

Intuition: High-profile talent often drives audience interest and marketing reach, potentially boosting revenue.

Objective: Create numerical features that capture both the presence of recognized industry figures (directors, writers) and the cumulative prominence of lead actors.

Modeling Approaches

Linear Modeling

We employed several linear modeling techniques:

- **Ordinary Least Squares (OLS)**: Ordinary Least Squares (OLS) is the foundational linear regression technique. It fits a hyperplane that minimizes the sum of squared residuals (differences between observed and predicted values). OLS is prized for its simplicity and interpretability—each coefficient represents the expected change in the target (revenue) for a one-unit change in the predictor, holding others constant. However, OLS assumes:
- **Elastic Net Regression**: Combination of L1 and L2 regularization, effectively handling multicollinearity.

Nonlinear Models

Advanced nonlinear techniques included:

- **Generalized Additive Models (GAM)**: Extended linear modeling using spline functions for capturing nonlinear effects.
- **Random Forest Regressor**: Random Forest constructs an ensemble of decision trees on bootstrapped samples, each using random subsets of features at splits. By averaging predictions across many trees, it reduces variance and prevents overfitting. Its non-parametric nature allows it to automatically capture complex nonlinearities and interactions among predictors without explicit feature engineering. Moreover, it provides feature importance scores, helping identify the most influential variables.
- **XGBoost**: XGBoost (Extreme Gradient Boosting) builds trees sequentially, where each new tree corrects errors from the ensemble so far. It uses second-order gradient information and built-in regularization (L1 and L2) to enhance predictive power while mitigating overfitting. XGBoost is optimized for speed and can handle large datasets efficiently, making it a standard for structured data tasks requiring high accuracy.

Partial Dependence:

- Trailer Likes (log): Roughly linear positive effect—more trailer engagement reliably boosts revenue expectations.
- Budget (log): Nonlinear pattern—diminishing returns at mid-range budgets, then a sharp rise for very large budgets (blockbusters).
- Budget×Actor Interaction: A concave relationship—films with combined high budgets and star casts earn disproportionately more until a plateau.
- Release Year: Gradual decline for the most recent releases, possibly reflecting market saturation or pandemic effects.

Takeaway:

GAM captures nuanced, nonlinear effects that OLS and Elastic Net cannot. The partial plots guide data-driven decisions—e.g., optimal budget ranges and year effects.

Heteroskedasticity:

Residuals fan out at higher fitted values. Next: Implement weighted least squares or use robust standard errors (HC3) in OLS.

Result

In this section, we summarize the implemented modeling approaches, their performance outcomes, diagnostic evaluations, and clear next steps for further enhancement.

Linear Modeling Results

Ordinary Least Squares (OLS) Regression

- **RMSE:** 9.85 Million
- **R-Squared:** 0.83

- **Cross-validated RMSE:** 19.43 Million
- **Cross-validated R-Squared:** 0.79

Violations (e.g., heteroskedasticity, collinearity) can lead to biased estimates or inflated variances.

Target: Log-transformed revenue, back-transformed for RMSE/R-Squared computation

Validation: 5-fold KFold (chronological, no shuffle), using timeseriessplit().

Diagnostic: Residuals vs Fitted shows clear heteroskedasticity (variance increases with fitted revenue).

Q-Q plot exhibits heavy right tail—extreme overestimation for blockbusters.

Histogram of residuals is sharply peaked with heavy tails.

Elastic Net Regression

- **RMSE**: 10.07 Million
- **R-Squared**: 0.82

Insights:

- Maintained interpretability: no coefficients driven to exact zero due to moderate L1 ratio.
- Elastic Net balanced accuracy and interpretability effectively amidst moderate multicollinearity.

Non-Linear Modeling Results

Generalized Additive Model (GAM)

- **RMSE**: 9.66 Million
- **R-Squared**: 0.84

Key Nonlinear Effects based on the graph:

- **trailer_pca**: Linear positive impact; increased trailer engagement reliably raises revenue.
- **Budget (log)**: Exhibits diminishing returns at mid-range budgets, significant boost for blockbuster budgets.
- **Budget×Actor Interaction**: Concave shape; significant increases up to a plateau, indicating budget and star power synergy.
- **Release Year**: Gradual decline for recent years, possibly market saturation or pandemic influences.

GAM identified nuanced nonlinear patterns, highlighting complex interactions and effects unseen in OLS and Elastic Net.

Random Forest Regressor

- **RMSE**: 9.64 Million
- **R-Squared**: 0.8371

Key Insights:

- Polynomial interactions and tuned hyperparameters improved capture of complex data relationships.
- Feature importance analysis strongly highlights PCA-derived trailer metrics as crucial predictors.

XGBoost with Time-based Data Split

- **RMSE**: 9.45 Million
- **R-Squared**: 0.8548

Limitations: Computational constraints limited hyperparameter tuning and feature optimization. Significant performance improvements remain achievable with expanded computing resources.

SQL Connection

We successfully implemented a MySQL database, which allows our data to benefit from its Atomicity, Consistency, Isolation, and Durability (ACID) properties. These properties help ensure that each transaction is reliable, errors don't leave the data in an inconsistent state, concurrent processes don't conflict, and changes are permanently saved once committed.

Beyond ACID, the database also improves performance and scalability. With a well-defined schema, data insertion follows strict rules, which helps maintain data integrity. We can easily export tables as CSV files for future use or offline analysis.

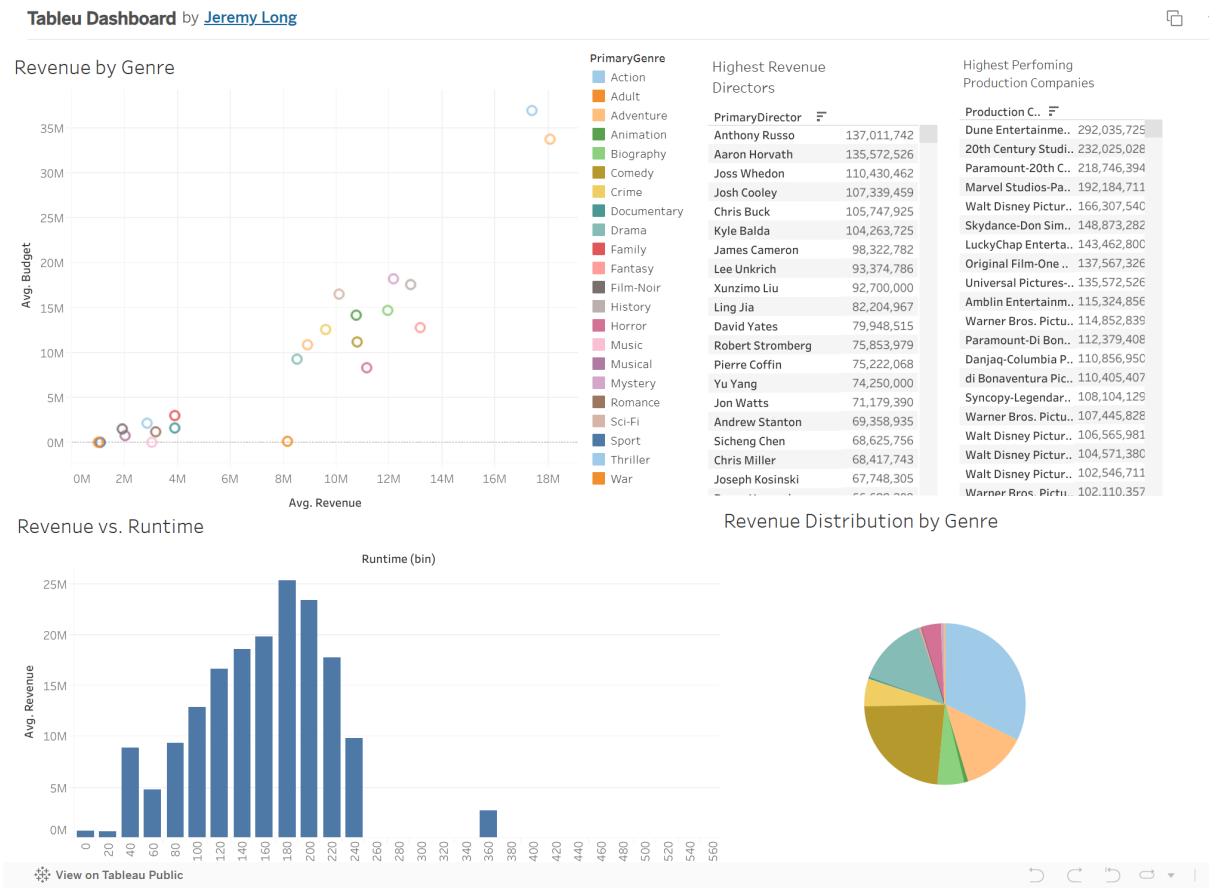
The database also integrates well with the website: the upload function checks for duplicate records, and we've built API endpoints for deleting and updating data. This makes it easy to manage content in real time without compromising accuracy or stability.

Website Navigation

By having a database connection, we implemented website functionality using the FastAPI framework. The FastAPI Python module allows us to quickly set up a basic functioning website that supports Create, Read, Update, and Delete (CRUD) endpoints.

To further enhance website functionality, we also experimented with CSS styling and JavaScript functions to toggle some of the API endpoints and other features, such as uploading new data and making predictions. We hope this will enhance the user experience with the prediction model.

Tabelau Dashboard



We decide to create a Tableau Dashboard for exploratory data analysis purpose and here's some interesting findings:

- Action and Adventure are the most popular (based on revenue) genres
- Movies seem to perform the best when it's runtime is around 180

Discussion

Across all methods—from OLS through GAM to Random Forest and XGBoost—we observed that increasing model flexibility yielded consistent gains in predictive accuracy. GAM achieved the highest R-squared (0.86) by capturing nuanced, nonlinear relationships (e.g., diminishing returns on mid-range budgets, synergy between budget and star power).

Challenges and Limitation

The main limitation in this project comes from the size and coverage of our secondary dataset. Although our primary IMDB dataset was large, most of our advanced features — such as trailer PCA components, star scores, and external metadata — were only available for a small subset of movies. This limited our final modeling dataset to a few thousand entries.

Because of this, we were unable to reliably train deep learning models; while we attempted them, the results showed high variance across runs, with performance shifting dramatically. This instability made them unsuitable for deployment or evaluation.

In addition, some of our engineered features (e.g., trailer embeddings via PCA, or our star scoring metric) rely on assumptions or incomplete data. In cases where trailers were missing or actor data was sparse, those features may have introduced noise rather than value.

Summary and Next Steps

Key Insights:

- Interpretability vs. Flexibility: OLS and Elastic Net yield straightforward coefficients, while GAM uncovers nonlinear patterns.
- Heteroskedasticity remains in all linear models; addressing it could reduce RMSE further.
- Outliers (blockbusters) drive error—targeted down-weighting or robust loss functions may help.

Next Steps

- Heteroskedasticity Correction: Implement Weighted Least Squares or robust standard errors in OLS/Elastic Net to stabilize error variance for blockbusters.

  Outlier Management: Compute Cook's distance to identify influential observations; apply Winsorization or conditional trimming to mitigate distortion.

  GAM Optimization: Tune the number and placement of spline knots for each predictor to push R-squared beyond 0.86.

  Enhanced Tree Tuning: Conduct an extensive RandomizedSearchCV on XGBoost parameters (n_estimators, max_depth, learning_rate, subsample) with increased computational resources.

  Feature Expansion: Integrate social-media sentiment and search-trend data, as well as new interaction terms (e.g., trailer engagement $\times$ release month) into both linear and tree-based models.

  Robust Deployment: Finalize FastAPI endpoints and SQL integration for real-time inference, and enhance the frontend to allow scenario testing and dynamic visualizations in the Tableau dashboard.

Conclusion

We have demonstrated a comprehensive workflow—spanning data collection, feature engineering, and a suite of predictive models—that reliably forecasts movie revenues within an inflation-adjusted, time-ordered paradigm. By highlighting the critical role of digital engagement alongside traditional metadata, our work equips studios with actionable insights to optimize both creative and marketing investments for future releases.

Reference

- IMDB. (2025). IMDb Non-Commercial Datasets. IMDB Developer. https://developer.imdb.com/non-commercial-datasets/